دليل JavaScript الشامل

دليل تعليمي شامل باللغة العربية

تاريخ الإنشاء: ٢٠٢٥/١١/٨

جدول المحتويات

1. مقدمة إلى جافاسكريبت

2. المتغيرات وأنواع البيانات

3. العوامل والتعبيرات

4. تدفق التحكم

5. الحلقات التكرارية والتكرار

6. الدوال

7. المصفوفات والكائنات

8. معالجة الأخطاء

9. جافاسكريبت غير المتزامن

10. التعامل مع نموذج كائن المستند

11. ميزات جافاسكريبت الحديثة

12. أفضل الممارسات وتصحيح الأخطاء

1. مقدمة إلى جافاسكريبت

مقدمة إلى جافاسكريبت

جافاسكريبت (JavaScript) هي لغة برمجة قوية ومتعددة الاستخدامات، تُعرف بكونها "لغة الويب". تستخدم بشكل أساسي لجعل صفحات الويب تفاعلية وديناميكية، ولكنها توسعت لتشمل تطوير تطبيقات الخادم (Node.js) وتطبيقات الجوال والديسكتوب أيضًا، مما يجعلها ضرورية لكل مطور ويب.

المتغيرات والإخراج إلى الشاشة

المتغيرات هي حاويات لتخزين البيانات. في جافاسكريبت، يمكنك تعريف متغير باستخدام `let` أو `const` أو `var`. لتصحيح الأخطاء أو لإظهار معلومات معينة في متصفح الويب (تحديداً في لوحة تحكم المطور)، نستخدم الدالة `console.log()`.

مثال برمجي (نظيف)

```
let greeting = "مرحباً بكم في عالم جافاسكريبت!";
let currentYear = 2024;

console.log(greeting);
console.log("نحن في عام: " + currentYear);
```

مثال برمجي (مع تعليقات عربية)

```
// تعريف متغير باسم 'greeting' لتخزين رسالة ترحيب نصية
let greeting = "مرحباً بكم في عالم جافاسكريبت!";

// تعريف متغير باسم 'currentYear' لتخزين رقم يمثل السنة الحالية
let currentYear = 2024;

// استخدام الدالة console.log() لعرض قيمة متغير 'greeting' في لوحة تحكم المتصفح
console.log(greeting);

// استخدام console.log() لعرض نص مدمج مع قيمة متغير 'currentYear' في لوحة تحكم المتصفح
console.log("نحن في عام: " + currentYear);
```

نقاط رئيسية

- جافاسكريبت هي لغة أساسية لإضفاء التفاعل والحيوية على صفحات الويب.

- تُستخدم المتغيرات (مثل `let`) لتخزين أنواع مختلفة من البيانات والمعلومات.
- الدالة `console.log()` أداة مهمة للمطورين لعرض المخرجات وتصحيح الأخطاء بسهولة.

---

2. المتغيرات وأنواع البيانات

المتغيرات وأنواع البيانات في JavaScript

المتغيرات هي حاويات لتخزين البيانات في برامجك. فهم كيفية تعريفها واستخدامها، ومعرفة الأنواع المختلفة للبيانات التي يمكنها حملها، هما أساسان لا غنى عنهما لأي مطور JavaScript.

المتغيرات (Variables)

المتغيرات تسمح لك بتخزين قيم يمكن استخدامها وتعديلها لاحقاً. في JavaScript، نستخدم الكلمات المفتاحية **`let`** و **`const`** لتعريف المتغيرات. استخدم **`let`** عندما تتوقع أن تتغير قيمة المتغير، و **`const`** للقيم الثابتة التي لا تتغير.

أنواع البيانات (Data Types)

تحدد أنواع البيانات نوع القيمة التي يخزنها المتغير. تتضمن JavaScript أنواع بيانات بدائية (Primitive Types) مثل:

- **`String`**: للنصوص (مثال: "مرحباً بالعالم").
- **`Number`**: للأرقام (سواء كانت صحيحة أو عشرية، مثال: 10 أو 3.14).
- **`Boolean`**: للقيم المنطقية (`true` أو `false`).

وهناك أنواع أخرى مثل `null` و `undefined`.

مثال على الكود (نظيف)

```
let userName = "أحمد";
const userAge = 30;
let isActive = true;

console.log(userName);
console.log(userAge);
console.log(isActive);

'let' سارة"; // إعادة تعيين لمتغير" = userName
```

```
console.log(userName);
```

مثال على الكود (مع تعليقات عربية)

```
// تعريف متغير باستخدام 'let' (يمكن تغيير قيمته لاحقاً) لتخزين اسم المستخدم
let userName = "أحمد";
// تعريف ثابت باستخدام 'const' (لا يمكن تغيير قيمته) لتخزين عمر المستخدم
const userAge = 30;
// تعريف متغير بولياني باستخدام 'let' لتخزين حالة النشاط
let isActive = true;

// طباعة قيمة المتغير userName إلى وحدة التحكم
console.log(userName);
// طباعة قيمة الثابت userAge إلى وحدة التحكم
console.log(userAge);
// طباعة قيمة المتغير isActive إلى وحدة التحكم
console.log(isActive);

// إعادة تعيين قيمة المتغير userName لأنه معرّف بـ 'let'
userName = "سارة";
// طباعة القيمة الجديدة لـ userName بعد التعديل
console.log(userName);
```

نقاط رئيسية

- تُستخدم المتغيرات لتخزين البيانات وتُعرف باستخدام **`let`** أو **`const`**.
- **`let`** للمتغيرات القابلة للتغيير، و **`const`** للثوابت التي لا تتغير قيمتها.
- تحدد أنواع البيانات مثل **`String`** و **`Number`** و **`Boolean`** نوع القيمة المخزنة.

---

3. العوامل والتعبيرات

العوامل والتعبيرات في JavaScript

في برمجة JavaScript، تعد **العوامل (Operators)** و**التعبيرات (Expressions)** من اللبنات الأساسية لكتابة منطق البرنامج. العوامل هي رموز خاصة تقوم بإجراء عمليات على القيم والمتغيرات، بينما التعبيرات هي أي جزء من الكود ينتج عنه قيمة واحدة عند تقييمه.

مفهوم العوامل (Operators)

العوامل هي رموز خاصة تخبر مترجم JavaScript بإجراء عملية معينة. توجد أنواع مختلفة منها، مثل العوامل الحسابية (+، -، *، /) لإجراء العمليات الرياضية، وعوامل المقارنة (==، ===، <، >) للمقارنة بين القيم، وعوامل الإسناد (=) لتعيين القيم للمتغيرات.

مفهوم التعبيرات (Expressions)

التعبير هو أي جزء من الكود ينتج عنه قيمة. يمكن أن يكون التعبير بسيطًا (مثل رقم `10` أو متغير `x`) أو معقدًا (مثل `x + y * 2`). عندما يتم "تقييم" تعبير ما، فإنه ينتج قيمة معينة يمكن استخدامها لاحقاً في البرنامج.

مثال عملي (كود نظيف)

```
let num1 = 10;
let num2 = 5;

let sum = num1 + num2;
let result = (num1 * 2) - num2;

console.log(sum);
console.log(result);
```

مثال عملي (مع تعليقات عربية)

```
// تعريف متغيرين باستخدام عامل الإسناد (=)
let num1 = 10;
let num2 = 5;

// هذا السطر يمثل تعبيراً يستخدم عامل الجمع (+)
// قيمة "num1" و "num2" تُجمع (num1 + num2) تُسند إلى المتغير (15) والنتيجة "sum"
let sum = num1 + num2; // هنا، "num1 + num2" 15 قيمة بسيط ينتج عنه تعبير هو

// هذا السطر يمثل تعبيراً أكثر تعقيداً
// يتم أولاً ضرب "num1" في 2 (عامل الضرب *)، ثم يتم طرح "num2" (عامل الطرح -)
// تُسند إلى المتغير (15) والنتيجة النهائية "result"
sult = (num1 * 2) - num2; // "(num1 * 2) - num2" 15 قيمة ينتج عنه تعبير معقد هو

// عرض النتائج في وحدة التحكم
```

```
console.log(sum);     // سيعرض 15
console.log(result); // سيعرض 15
```

- **العوامل (Operators)**: رموز خاصة تقوم بإجراء عمليات محددة (مثل الجمع، المقارنة، الإسناد).
- **التعبيرات (Expressions)**: أي جزء من الكود يتم تقييمه لينتج عنه قيمة واحدة.
- فهم العوامل والتعبيرات ضروري لبناء أي منطق برمجي فعال في JavaScript.

---

4. تدفق التحكم

تدفق التحكم في JavaScript

تدفق التحكم هو العمود الفقري لأي برنامج، فهو يحدد الترتيب الذي تُنفذ به التعليمات البرمجية. في JavaScript، يسمح لنا تدفق التحكم باتخاذ قرارات وتكرار مهام معينة بناءً على شروط محددة، مما يجعل برامجنا ديناميكية وذكية.

مفهوم تدفق التحكم

يتيح لك تدفق التحكم في JavaScript توجيه مسار تنفيذ الكود. يمكننا استخدام عبارات شرطية (مثل `if/else`) لتنفيذ كتل معينة من الكود فقط إذا تم استيفاء شرط معين، أو حلقات تكرارية (مثل `for/while`) لتكرار كتلة من الكود عدة مرات.

مثال برمجي (بدون تعليقات)

```
let age = 18;

if (age >= 18) {
console.log("أنت مؤهل للتصويت.");
} else {
console.log("أنت غير مؤهل للتصويت بعد.");
}
```

مثال برمجي (مع تعليقات عربية)

```
// تعريف متغير لتخزين العمر
let age = 18;

// التحقق من الشرط: إذا كان العمر أكبر من أو يساوي 18
if (age >= 18) {
// تنفيذ هذا الكود إذا كان الشرط صحيحًا
```

```
console.log("أنت مؤهل للتصويت.");
} else {
// تنفيذ هذا الكود إذا كان الشرط خاطئًا
console.log("أنت غير مؤهل للتصويت بعد.");
}
```

نقاط رئيسية

- **العبارات الشرطية:** (مثل `if, else if, else`) تُستخدم لاتخاذ القرارات وتنفيذ كود محدد بناءً على شروط.
- **الحلقات التكرارية:** (مثل `for, while, do-while`) تُستخدم لتكرار تنفيذ كتل من الكود عدة مرات.
- يُعد تدفق التحكم أساسيًا لبناء برامج منطقية تتفاعل بذكاء مع البيانات والمدخلات.

---

5. الحلقات التكرارية والتكرار

الحلقات التكرارية والتكرار في JavaScript

تُعدّ الحلقات التكرارية (Loops) من الأدوات الأساسية في البرمجة التي تُمكّنك من تنفيذ كتلة من التعليمات عدة مرات تلقائيًا دون الحاجة إلى كتابتها مرارًا وتكرارًا. هذا يوفر الوقت ويجعل الكود أكثر كفاءة وسهولة في الصيانة.

مفهوم الحلقات التكرارية (For Loop)

تسمح لك حلقة `for` بتكرار جزء من الكود عددًا محددًا من المرات. تُستخدم عادةً عندما تعرف مسبقًا عدد مرات التكرار المطلوبة. تتكون من ثلاثة أجزاء رئيسية: التهيئة، الشرط، وزيادة أو نقصان العداد.

مثال برمجي (كود نظيف)

```
for (let i = 1; i <= 5; i++) {
    console.log("العدد هو: " + i);
}
```

مثال برمجي (مع تعليقات عربية)

```
// تعريف حلقة 'for' للتكرار من 1 إلى 5
for (let i = 1; // تهيئة: يبدأ العداد 'i' من القيمة 1
     i <= 5; // الشرط: تستمر الحلقة طالما 'i' أقل من أو يساوي 5
     i++ // التحديث: زيادة 'i' بمقدار 1 بعد كل تكرار
    ) {
    // الكود الذي سيتم تنفيذه في كل مرة من التكرار
```

```
    console.log("العدد هو: " + i); // طباعة رسالة تحتوي على قيمة العداد الحالية
}
```

نقاط رئيسية

- تُستخدم الحلقات التكرارية لأتمتة المهام المتكررة وتجنب تكرار الكود.
- تُعد حلقة `for` مثالية عندما يكون عدد مرات التكرار معروفًا مسبقًا.
- الحلقات تزيد من كفاءة البرامج وتجعلها أكثر ديناميكية.

---

6. الدوال

الدوال في JavaScript

الدوال (Functions) هي كتل برمجية مصممة لأداء مهمة معينة. تسمح لك بكتابة جزء من الكود مرة واحدة واستخدامه عدة مرات في برنامجك، مما يجعل الكود منظمًا وسهل الصيانة.

لماذا نستخدم الدوال؟

تساعد الدوال في تجنب تكرار الكود (DRY - Don't Repeat Yourself). إنها تجعل برنامجك أكثر قابلية للقراءة، وتسهل تصحيح الأخطاء، وتحسن من تنظيم المشروع بتقسيم المهام الكبيرة إلى وحدات أصغر.

مثال عملي (كود نظيف)

```
function greet(name) {
return "أهلاً بك يا " + name + "!";
}

let message = greet("علي");
console.log(message);
```

مثال عملي (مع تعليقات عربية)

```
// تعريف دالة باسم 'greet' تستقبل معاملًا واحدًا هو 'name'
function greet(name) {
// تعيد الدالة نصًا ترحيبيًا باستخدام قيمة 'name'
return "أهلاً بك يا " + name + "!";
```

```
}

// استدعاء الدالة 'greet' للمعامل كقيمة "علي" مع تمرير 'name'
// تخزين القيمة المعادة في المتغير 'message'
let message = greet("علي");

// طباعة قيمة المتغير 'message' إلى وحدة التحكم
console.log(message);
```

نقاط رئيسية

- **إعادة الاستخدام:** تسمح لك الدوال بإعادة استخدام نفس كتلة الكود عدة مرات في أماكن مختلفة.
- **تنظيم الكود:** تجعل الكود أكثر تنظيمًا ووضوحًا بتقسيم المهام الكبيرة إلى أجزاء أصغر يمكن إدارتها.
- **المدخلات والمخرجات:** يمكن للدوال استقبال قيم (تسمى "معاملات" أو "وسائط") ومعالجة هذه القيم لإنتاج نتائج.

---

7. المصفوفات والكائنات

المصفوفات والكائنات في JavaScript

تُعد **المصفوفات (Arrays)** و**الكائنات (Objects)** من الهياكل الأساسية لتخزين البيانات وتنظيمها في JavaScript. فبينما تُستخدم المصفوفات لتخزين قوائم مرتبة من العناصر، تُستخدم الكائنات لتمثيل بيانات أكثر تعقيدًا في شكل أزواج من المفاتيح والقيم.

المصفوفات (Arrays)

المصفوفة هي قائمة مرتبة من القيم. يتم الوصول إلى عناصرها باستخدام فهرس رقمي يبدأ من الصفر. يمكن للمصفوفة أن تحتوي على أنواع بيانات مختلفة.

الكائنات (Objects)

الكائن هو مجموعة من الخصائص، حيث كل خاصية عبارة عن زوج من "المفتاح" (key) و"القيمة" (value). المفاتيح عادةً ما تكون سلاسل نصية، والقيم يمكن أن تكون أي نوع بيانات.

مثال برمجي (بدون تعليقات)

```
const fruits = ["تفاح", "برتقال", "موز"];
const person = {
```

```
name: "أحمد",
age: 30,
city: "الرياض"
};

console.log(fruits[0]);
console.log(person.name);
```

مثال برمجي (مع تعليقات عربية)

```
// تعريف مصفوفة (Array) تحتوي على أسماء فواكه
const fruits = ["تفاح", "برتقال", "موز"];

// تعريف كائن (Object) يمثل شخص بخصائص مختلفة
const person = {
name: "المفتاح هو: // خاصية الاسم: أحمد",  "name" القيمة هي "أحمد"
age: 30,  // المفتاح هو: خاصية العمر: 30 "age" والقيمة هي
city: "المفتاح هو: // خاصية المدينة: الرياض"  "city" القيمة هي "الرياض"
};

// الوصول إلى العنصر الأول في المصفوفة باستخدام الفهرس (index)
console.log(fruits[0]); // "سيُخرج: "تفاح

// الوصول إلى قيمة خاصية في الكائن باستخدام نقطة (.) "name"
console.log(person.name); // "سيُخرج: "أحمد
```

خلاصة:

- المصفوفات تُنظّم البيانات في قوائم مرتبة ومفهرسة رقميًا.
- الكائنات تُنظّم البيانات كأزواج مفتاح-قيمة لتمثيل كيانات ذات خصائص.
- كلاهما أدوات أساسية وقوية لإدارة البيانات في برامج JavaScript.

---

8. معالجة الأخطاء

معالجة الأخطاء في JavaScript

تعلم كيفية التعامل مع الأخطاء أمر ضروري لإنشاء تطبيقات JavaScript قوية وموثوقة. تتيح لك معالجة الأخطاء اكتشاف المشكلات المحتملة والتعامل معها بلطف، بدلاً من إيقاف تشغيل البرنامج فجأة.

مفهوم try...catch

جملة `try...catch` هي البنية الأساسية لمعالجة الأخطاء. يوضع الكود الذي قد يتسبب في خطأ داخل كتلة `try`، وإذا حدث خطأ، يتم "التقاطه" في كتلة `catch` حيث يمكن معالجته وعرض رسالة مناسبة للمستخدم.

مثال عملي (كود نظيف)

```
function divide(a, b) {
  if (b === 0) {
    throw new Error("لا يمكن القسمة على صفر.");
  }
  return a / b;
}

try {
  let result = divide(10, 0);
  console.log("النتيجة:", result);
} catch (error) {
  console.error("حدث خطأ:", error.message);
}
```

مثال عملي (مع تعليقات عربية)

```
function divide(a, b) {
  // التحقق مما إذا كان المقسوم عليه صفرًا
  if (b === 0) {
    // إذا كان صفرًا، قم بإلقاء خطأ جديد (throw)
    throw new Error("لا يمكن القسمة على صفر.");
  }
  // إذا كان المقسوم عليه غير صفر، أعد نتيجة القسمة
  return a / b;
}

// لاحتواء الكود الذي قد يتسبب في خطأ try كتلة
```

```
try {
  // (هنا ستلقي خطأ لأننا نقسم على صفر) ستدعاء الدالة التي قد تلقي خطأ
  let result = divide(10, 0);
  // هذا السطر لن يتم تنفيذه إذا حدث خطأ داخل كتلة try
  console.log("النتيجة:", result);
} catch (error) {
  // كتلة catch في كتلة try لالتقاط الخطأ إذا تم إلقاؤه
  // error هو الكائن الذي يمثل الخطأ الذي تم التقاطه
  console.log("حدث خطأ في العملية:", error.message);
}
```

نقاط رئيسية

- تُستخدم **`try...catch`** لالتقاط الأخطاء ومعالجتها بشكل أنيق ومنظم.
- تُلقى الأخطاء باستخدام **`throw new Error(...)`** لإيقاف التنفيذ وإرسال الخطأ إلى كتلة `catch`.
- تساعد معالجة الأخطاء في جعل تطبيقاتك أكثر **ثباتًا وسهولة في الاستخدام** من خلال توفير رسائل خطأ واضحة أو بدائل آمنة.

---

9. جافاسكريبت غير المتزامن

جافاسكريبت غير المتزامن (Asynchronous JavaScript)

في جافاسكريبت، تعني العمليات المتزامنة (synchronous) أن المهام تُنفذ بالتسلسل، واحدة تلو الأخرى، مما قد يؤدي إلى "تجميد" واجهة المستخدم إذا كانت هناك مهمة طويلة. جافاسكريبت غير المتزامن (asynchronous) يسمح بتنفيذ مهام معينة في الخلفية دون حظر (blocking) الكود الرئيسي، مما يحافظ على استجابة التطبيق.

مفهوم جافاسكريبت غير المتزامن

يمكّن جافاسكريبت غير المتزامن برنامجك من بدء مهمة تتطلب وقتًا طويلاً (مثل جلب البيانات من الخادم أو مؤقت) والاستمرار في تنفيذ الكود الآخر بدلاً من الانتظار. عندما تنتهي المهمة الطويلة، يتم إعلام الكود الخاص بك. الأدوات الرئيسية لهذه العمليات هي الدوال المرتجعة (Callbacks)، الوعود (Promises)، وكلمات

`async/await`

.

مثال الكود (نظيف)

```
console.log("البدء");

setTimeout(() => {
console.log("تم تنفيذ العملية غير المتزامنة بعد 2 ثانية");
}, 2000);

console.log("الانتهاء");
```

مثال الكود (مع تعليقات عربية)

```
// هذا السطر سينفذ أولاً بشكل متزامن وفوري
console.log("البدء");

// هذه دالة غير متزامنة (setTimeout)
// ستنتظر 2000 مللي ثانية (2 ثانية) قبل تنفيذ الدالة بداخلها
setTimeout(() => {
// هذا السطر سينفذ بعد مرور 2 ثانية،
// لكنه لن يمنع الكود الذي يليه من التنفيذ فوراً
console.log("تم تنفيذ العملية غير المتزامنة بعد 2 ثانية");
}, 2000); // المدة الزمنية بالمللي ثانية

// هذا السطر سينفذ فوراً بعد دالة setTimeout،
// أي قبل أن يتم تنفيذ الكود بداخل setTimeout
console.log("الانتهاء");
```

نقاط رئيسية

- **تحسين التجربة:** يسمح بتنفيذ المهام دون حظر واجهة المستخدم، مما يحافظ على تفاعلية التطبيق.
- **المهام الطويلة:** يستخدم للتعامل مع العمليات التي تستغرق وقتًا طويلاً مثل طلبات الشبكة، قراءة الملفات، والمؤقتات.
- **الأدوات:** الأدوات الرئيسية للتعامل مع العمليات غير المتزامنة تشمل الدوال المرتجعة (Callbacks)، الوعود (Promises)، و `async/await`.

10. التعامل مع نموذج كائن المستند

التعامل مع نموذج كائن المستند (DOM) في JavaScript

نموذج كائن المستند (DOM) هو واجهة برمجة تطبيقات (API) للمستندات HTML و XML. يتيح DOM للغة JavaScript الوصول إلى محتوى صفحات الويب وتعديله وتغيير هيكلها وتنسيقها بشكل ديناميكي، مما يجعل الصفحات تفاعلية.

ما هو DOM ولماذا هو مهم؟

يعمل DOM كتمثيل شجري (Tree-Structure) لصفحة الويب، حيث يمثل كل جزء من المستند (مثل العناصر والنصوص والسمات) "عقدة" (Node). يمكّن هذا التمثيل JavaScript من التعامل مع هذه العقد لتحديث الواجهة الرسومية للمستخدم (UI) استجابةً لأحداث معينة أو لتغيير البيانات المعروضة دون الحاجة لإعادة تحميل الصفحة بالكامل.

مثال برمجي (رمز نظيف)

```
const paragraph = document.getElementById('myParagraph');
paragraph.textContent = 'تم تغيير هذا النص بواسطة JavaScript!';
paragraph.style.color = 'blue';
paragraph.classList.add('highlight');
```

مثال برمجي (مع تعليقات عربية)

```
// فرضاً لدينا في HTML: <p id="myParagraph">نص أصلي</p>
'myParagraph' باستخدام معرفه <p> حدد العنصر // ;('myParagraph')t.getElementById
غيّر المحتوى النصي للعنصر // ;'تم تغيير هذا النص بواسطة JavaScript!' = xtContent
عيّن لون النص ليكون أزرق // ;'paragraph.style.color = 'blue
للعنصر 'highlight' باسم CSS أضف فئة // ;('highlight')paragraph.classList.add
```

نقاط رئيسية

- **تمثيل شجري:** DOM هو تمثيل كائني وشجري لصفحة الويب (HTML)، يسهل الوصول إلى أجزائها.
- **التفاعل الديناميكي:** يتيح JavaScript التفاعل الديناميكي مع DOM لتحديث محتوى وتصميم الصفحة.
- **العمليات الأساسية:** تشمل عمليات DOM الشائعة: تحديد العناصر، تغيير المحتوى، إضافة/إزالة الفئات، وتعديل الأنماط.

11. ميزات جافاسكريبت الحديثة

ميزات جافاسكريبت الحديثة

تتطور لغة جافاسكريبت باستمرار، مقدمةً ميزات جديدة تجعل كتابة الكود أسهل وأكثر كفاءة وقابلية للقراءة. هذه الميزات تساعد المطورين على بناء تطبيقات ويب عصرية وقوية وتحسين تجربة البرمجة.

قوالب النصوص الحرفية (Template Literals)

تُعد قوالب النصوص الحرفية (المعروفة أيضاً بـ "String Templates") إحدى الميزات القوية التي تبسط التعامل مع السلاسل النصية. تسمح لك بإنشاء سلاسل نصية متعددة الأسطر وبدمج المتغيرات أو التعبيرات داخلها بسهولة باستخدام علامات الاقتباس الخلفية (` `) وعلامة الدولار والقوسين المعقوفين ${}.

مثال برمجي (كود نظيف)

```
const name = "أحمد";
const age = 30;
const greeting = `اسمي ${name} مرحباً، وعمري ${age} سنة.`;
console.log(greeting);
```

مثال برمجي (مع تعليقات عربية)

```
// تعريف متغير لتخزين الاسم باستخدام 'const'
const name = "أحمد";
// تعريف متغير لتخزين العمر
const age = 30;

// لإنشاء رسالة ترحيب (Template Literal) استخدام قالب نصي حرفي
// ${} مباشرة داخل السلسلة باستخدام (name, age) يتيح لنا دمج المتغيرات
const greeting = `اسمي ${name} مرحباً، وعمري ${age} سنة.`;

// طباعة الرسالة الناتجة إلى وحدة التحكم
console.log(greeting);
```

نقاط رئيسية

- **تحسين قراءة الكود:** تجعل السلاسل النصية المعقدة أسهل في الفهم والصيانة.
- **دمج المتغيرات بسهولة:** يمكن تضمين المتغيرات والتعبيرات مباشرة باستخدام بناء ${}.

- **دعم الأسطر المتعددة:** تسمح بكتابة سلاسل نصية تمتد على عدة أسطر دون الحاجة لرموز خاصة مثل `\n`.

---

12. أفضل الممارسات وتصحيح الأخطاء

أفضل الممارسات وتصحيح الأخطاء في JavaScript

تُعد كتابة كود JavaScript نظيف وفعال وقابل للصيانة، بالإضافة إلى القدرة على تحديد وإصلاح الأخطاء بسرعة، مهارات أساسية لأي مطور. هذه الممارسات تضمن جودة الكود واستقراره.

ما هي أفضل الممارسات ولماذا هي مهمة؟

أفضل الممارسات هي مجموعة من الإرشادات والمعايير المتفق عليها التي تساعد في كتابة كود عالي الجودة. إنها تجعل الكود سهل القراءة والفهم، وتقلل من الأخطاء المحتملة، وتسهل على المطورين الآخرين (أو عليك في المستقبل) تعديله وصيانته بكفاءة.

أساسيات تصحيح الأخطاء (Debugging)

تصحيح الأخطاء هو عملية البحث عن الأخطاء (Bugs) في الكود وإصلاحها. في JavaScript، غالبًا ما نستخدم `console.log()` لتتبع قيم المتغيرات ومسار التنفيذ، بالإضافة إلى أدوات المطور المدمجة في المتصفح التي توفر إمكانيات قوية لتحديد المشاكل وتحليل الأداء.

مثال عملي (كود نظيف)

```
function calculateRectangleArea(width, height) {
    if (width <= 0 || height <= 0) {
        console.error("Width and height must be positive numbers.");
        return 0;
    }
    return width * height;
}

const rectWidth = 10;
const rectHeight = 5;
const area = calculateRectangleArea(rectWidth, rectHeight);
```

```
console.log(`Rectangle Area: ${area}`);
```

مثال عملي (مع تعليقات عربية)

```
// تعريف دالة لحساب مساحة المستطيل
// تستقبل الدالة معلمات (parameters) للعرض والارتفاع
function calculateRectangleArea(width, height) {
    // التحقق من صحة المدخلات: كأفضل ممارسة
    // يجب أن يكون العرض والارتفاع أكبر من صفر
    if (width <= 0 || height <= 0) {
        // استخدام console.error للإبلاغ عن خطأ واضح في المدخلات
        console.error("يجب أن يكون العرض والارتفاع أعدادًا موجبة.");
        return 0; // إرجاع صفر في حالة المدخلات غير الصالحة
    }
    // حساب وإرجاع مساحة المستطيل
    return width * height;
}

// تعريف متغيرات بأسماء واضحة ومفهومة (أفضل ممارسة)
const rectWidth = 10;
const rectHeight = 5;

// استدعاء الدالة وحفظ النتيجة في متغير
const area = calculateRectangleArea(rectWidth, rectHeight);

// استخدام console.log لعرض النتيجة أو لتصحيح الأخطاء (debugging)
console.log(`مساحة المستطيل: ${area}`);
```

نقاط أساسية

- **الكود النظيف:** استخدام أسماء واضحة للمتغيرات والدوال، وهيكل منطقي يسهل الفهم والصيانة.
- **التحقق من المدخلات:** التأكد من أن البيانات التي تستقبلها الدوال صحيحة لتجنب الأخطاء غير المتوقعة.
- **أدوات تصحيح الأخطاء:** `console.log()` وأدوات المطور في المتصفح ضرورية لتحديد المشاكل وإصلاحها بفاعلية.